

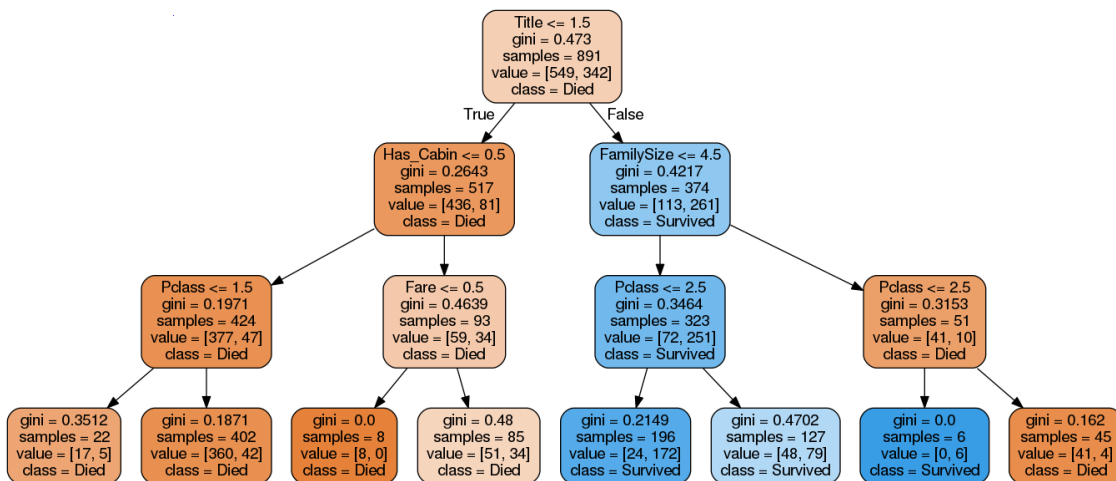
decision_tree

June 15, 2026

1 Decision Trees (Cây quyết định)

1.1 1. Giới thiệu (Introduction)

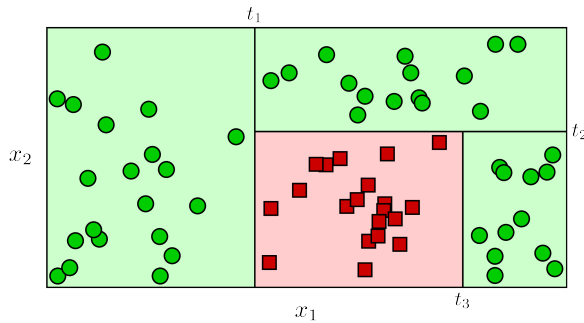
Decision Trees là một thuật toán học máy phổ biến, thuộc nhóm học có giám sát (supervised learning). Nó có thể được sử dụng cho cả bài toán phân loại (classification) và hồi quy (regression). Như tên gọi, thuật toán này hoạt động bằng cách tạo ra một mô hình dạng cây để đưa ra các quyết định. Mỗi nút trong cây biểu thị một thuộc tính hoặc một đặc trưng, mỗi nhánh biểu thị một quy tắc quyết định và mỗi lá cây biểu thị kết quả cuối cùng (nhãn lớp hoặc giá trị dự đoán).



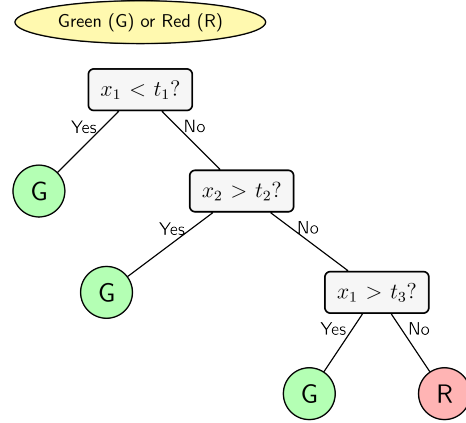
1.2 2. Lý thuyết (Theory)

1.2.1 2.1 Cách thức hoạt động cơ bản

Một Decision Tree hoạt động bằng cách chia tập dữ liệu thành các tập con nhỏ hơn dựa trên các câu hỏi ‘yes-no’ đơn giản. Quá trình này tiếp tục một cách đệ quy cho đến khi các tập con trở đạt đến một tiêu chí dừng nào đó.



(a)



(b)

Cấu trúc cây bao gồm:

- **Nút gốc:** Nút trên cùng của cây, biểu thị toàn bộ tập dữ liệu.
- **Nút bên trong:** Biểu thị một điều kiện kiểm tra trên một thuộc tính. Từ nút này, các nhánh được tạo ra dựa trên kết quả của điều kiện kiểm tra.
- **Nhánh:** Biểu thị các quy tắc quyết định hoặc các giá trị có thể có của thuộc tính được kiểm tra.
- **Nút lá :** Nút cuối cùng của cây, không có nhánh con nào. Nó biểu thị kết quả cuối cùng (nhãn lớp trong phân loại hoặc giá trị dự đoán trong hồi quy).

1.2.2 2.2 Các khái niệm quan trọng

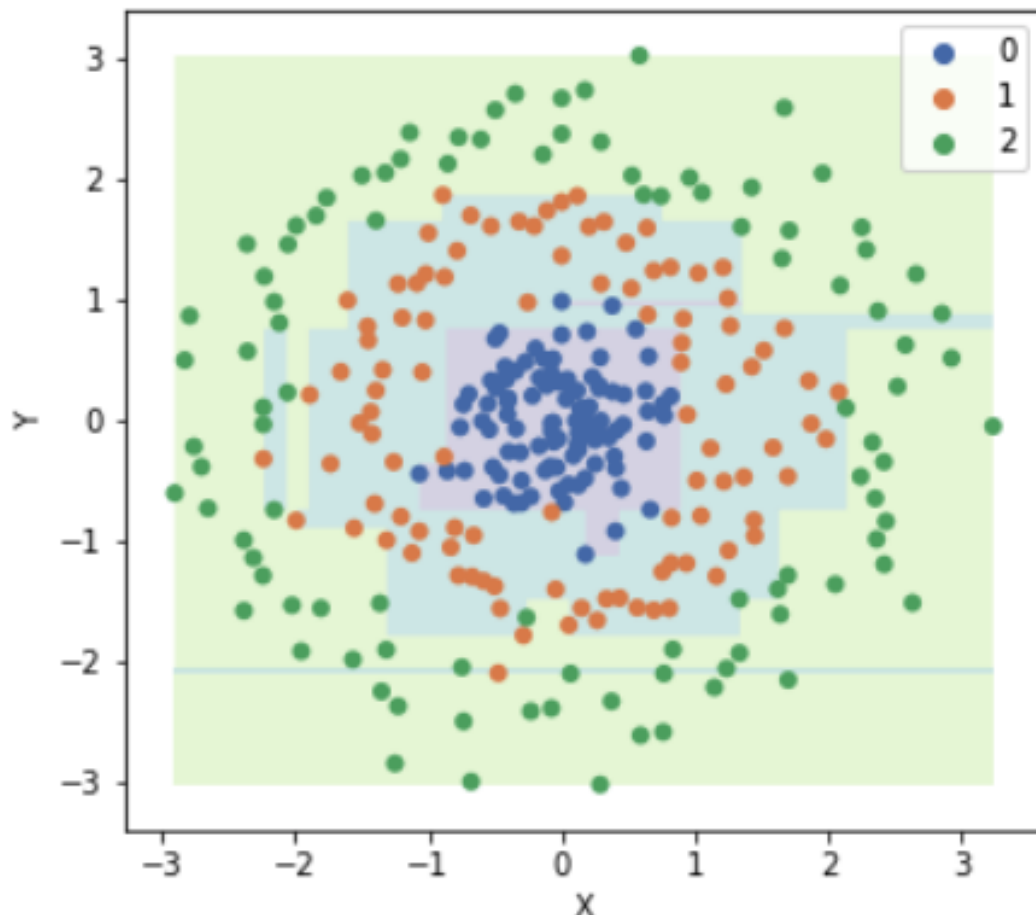
2.2.1 Độ thuần khiết (Purity) và Tiêu chí chia tách (Splitting Criteria) Để xây dựng cây, tại mỗi nút, thuật toán cần quyết định thuộc tính nào và giá trị nào để chia tách dữ liệu sao cho các tập con sau khi chia tách là ‘thuần khiết’ nhất có thể (tức là chứa chủ yếu các mẫu thuộc cùng một lớp).

Các tiêu chí phổ biến để đo lường độ thuần khiết và quyết định chia tách bao gồm:

- **Gini Impurity (Độ tạp Gini):** Gini impurity đo lường xác suất một phần tử được chọn ngẫu nhiên từ tập dữ liệu sẽ bị gán nhầm lớp nếu nó được gán ngẫu nhiên theo phân phối lớp trong tập con đó. Giá trị Gini impurity càng thấp thì tập con càng thuần khiết.
- **Entropy (Độ hỗn loạn/Độ bất định):** Entropy đo lường mức độ bất định hoặc hỗn loạn của một tập dữ liệu. Giá trị Entropy bằng 0 khi tập con hoàn toàn thuần khiết (chỉ chứa một lớp duy nhất) và đạt cực đại khi các lớp được phân bố đều.
- **Information Gain (Độ lợi thông tin):** Là sự giảm Entropy khi một tập dữ liệu được chia tách dựa trên một thuộc tính. Thuật toán chọn thuộc tính mang lại Information Gain lớn nhất để chia tách tại mỗi nút.

2.2.2 Overfitting Các thuật toán Decision Tree nói chung nếu xây dựng cây quyết định đủ sâu thì sẽ tách được các node lá chỉ chứa dữ liệu một lớp nhất định, nên mô hình rất dễ bị overfitting.

Overfitting là hiện tượng mô hình tìm được quá khớp với dữ liệu training. Việc quá khớp này có thể dẫn đến việc dự đoán nhầm lẫn, và chất lượng mô hình không còn tốt trên dữ liệu test nữa.



Để giải quyết vấn đề quá khớp này, ta có thể sử dụng: * Cắt tỉa cành (pruning). * Dùng thuật toán Random Forest.

1.3 3. Các bước tiến hành

1.3.1 3.1 Chuẩn bị dữ liệu (Data Preparation)

- **Thu thập dữ liệu:** Tập hợp dữ liệu phù hợp với bài toán của bạn (phân loại hoặc hồi quy).
- **Tiền xử lý dữ liệu:**
 - **Xử lý thiếu giá trị (Missing Values):** Điền các giá trị thiếu (imputation) bằng giá trị trung bình, trung vị, mode hoặc loại bỏ các hàng/cột có giá trị thiếu.
 - **Mã hóa biến phân loại (Categorical Encoding):** Decision Tree hoạt động tốt với cả dữ liệu số và phân loại, nhưng nhiều thư viện yêu cầu biến phân loại được mã hóa thành số (ví dụ: One-Hot Encoding, Label Encoding).
 - **Chia tách tập dữ liệu:** Chia dữ liệu thành tập huấn luyện (training set) và tập kiểm tra (test set). Tập huấn luyện dùng để xây dựng mô hình, và tập kiểm tra dùng để đánh giá hiệu suất của mô hình trên dữ liệu chưa từng thấy.

1.3.2 3.2 Xây dựng mô hình Decision Tree

Trong Python, thư viện `scikit-learn` cung cấp các công cụ mạnh mẽ để xây dựng Decision Tree.

```
[1]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

[2]: # Bước 1: Tải và chuẩn bị dữ liệu (Ví dụ với một tập dữ liệu giả định)
# Giả sử bạn có DataFrame 'df' với các cột đặc trưng (features) và cột mục tiêu
# ↪ (target)
# Ví dụ: Dữ liệu về khách hàng có mua sản phẩm không dựa trên tuổi, thu nhập, v.
# ↪ v.

data = {
    'Age': [25, 30, 35, 40, 45, 50, 55, 60, 28, 33],
    'Income': [50000, 60000, 70000, 80000, 90000, 100000, 110000, 120000,
    ↪ 55000, 65000],
    'Student': ['No', 'Yes', 'No', 'No', 'Yes', 'No', 'No', 'Yes', 'No', 'Yes'],
    'BuysProduct': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'Yes', 'No',
    ↪ 'Yes']
}
df = pd.DataFrame(data)

[3]: # Mã hóa biến phân loại 'Student' và 'BuysProduct'
df['Student'] = df['Student'].map({'No': 0, 'Yes': 1})
df['BuysProduct'] = df['BuysProduct'].map({'No': 0, 'Yes': 1})

X = df[['Age', 'Income', 'Student']] # Đặc trưng
y = df['BuysProduct'] # Biến mục tiêu

[4]: # Bước 2: Chia tách dữ liệu thành tập huấn luyện và tập kiểm tra
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    ↪ random_state=42)

[5]: # Bước 3: Khởi tạo và huấn luyện mô hình Decision Tree
# criterion có thể là 'gini' hoặc 'entropy'
# max_depth giới hạn độ sâu của cây để tránh overfitting
model = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
model.fit(X_train, y_train)

[5]: DecisionTreeClassifier(max_depth=3, random_state=42)
```

1.3.3 3.3 Đánh giá mô hình (Model Evaluation)

Sau khi huấn luyện, bạn cần đánh giá hiệu suất của mô hình trên tập kiểm tra để xem nó hoạt động tốt như thế nào. Các chỉ số đánh giá phổ biến bao gồm:

- `model.score(X_test, y_test)`: tỷ lệ chính xác / điểm của mô hình.

- `model.predict(X_test)`: dùng để dự đoán giá trị.

```
[6]: # Dự đoán trên tập kiểm tra
y_pred = model.predict(X_test)
print('Dự đoán:', y_pred)
print('Thực tế:', y_test.values)

# Đánh giá hiệu suất
print('Độ chính xác:', model.score(X_test, y_test))
```

Dự đoán: [0 1 1]

Thực tế: [0 0 0]

Độ chính xác: 0.3333333333333333

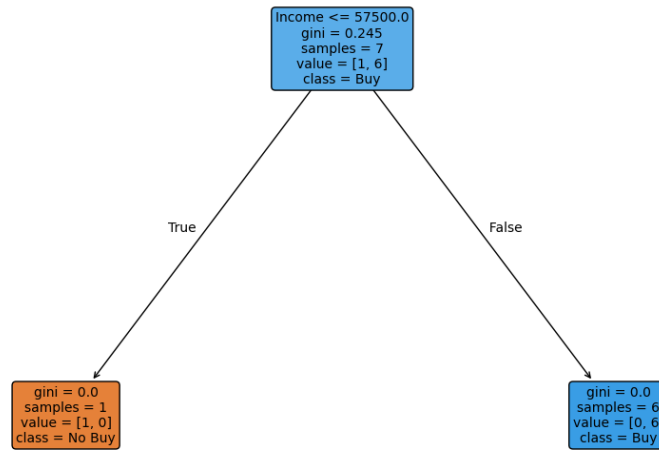
1.3.4 3.4 Trực quan hóa Decision Tree

Việc trực quan hóa cây quyết định giúp chúng ta hiểu rõ hơn về cách mô hình đưa ra các quyết định.

```
[7]: import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(15, 10))
plot_tree(model,
          feature_names=X.columns.tolist(),
          class_names=['No Buy', 'Buy'],
          filled=True,
          rounded=True,
          fontsize=10)
plt.title("Decision Tree Visualization")
plt.show()
```

Decision Tree Visualization



```
[8]: # ví dụ 2, xây dựng Decision Tree từ bộ dữ liệu iris
from sklearn.datasets import load_iris, load_wine

# 1. Tải bộ dữ liệu
iris = load_iris()
X, y = iris.data, iris.target

# 2. Tách dữ liệu
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
    random_state=42)

# 3. Khởi tạo và huấn luyện mô hình
clf = DecisionTreeClassifier(max_depth=2, random_state=42)
clf.fit(X_train, y_train)

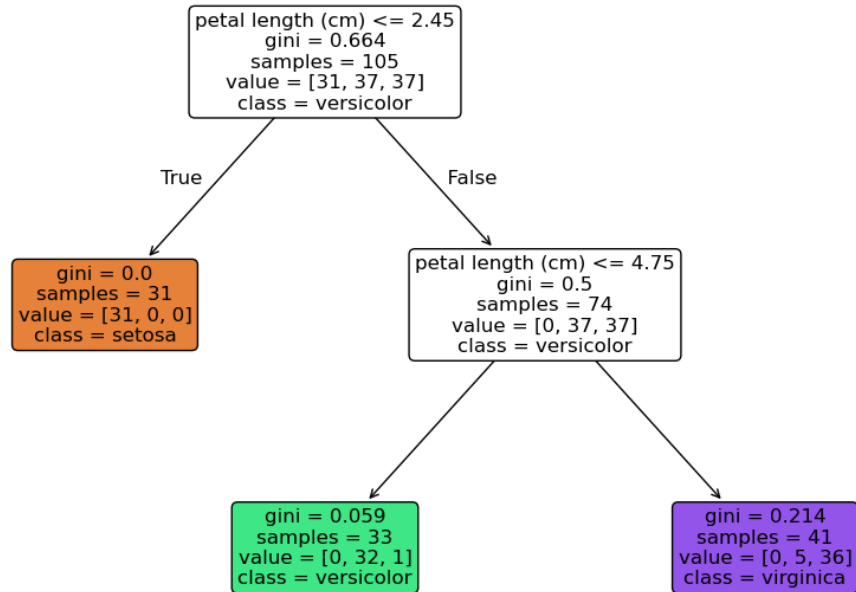
# 3. Biểu diễn mô hình
plt.figure(figsize=(12, 8))
plot_tree(clf,
    feature_names=iris.feature_names,
    class_names=iris.target_names.tolist(),
    filled=True,
    rounded=True,
```

```

        fontsize=12)
plt.title(f"Độ chính xác: {clf.score(X_test, y_test):.4f}")
plt.show();

```

Độ chính xác: 0.9778



1.4 4. Hạn chế (Limitations)

Mặc dù Decision Tree có nhiều ưu điểm như dễ hiểu, không cần chuẩn hóa dữ liệu và có khả năng xử lý cả dữ liệu số lẫn phân loại, nó vẫn tồn tại một số hạn chế quan trọng.

1.4.1 4.1 Dễ bị quá khớp (Overfitting)

Đây là nhược điểm lớn nhất. Nếu không được giới hạn độ sâu hoặc cắt tỉa (pruning), cây có thể phát triển quá chi tiết, học luôn cả nhiễu (noise) trong dữ liệu huấn luyện. Điều này dẫn đến việc mô hình hoạt động cực tốt trên tập huấn luyện nhưng rất kém trên dữ liệu thực tế.

1.4.2 4.2 Kém ổn định (Instability)

Decision Trees khá nhạy cảm. Chỉ một thay đổi nhỏ trong dữ liệu huấn luyện (ví dụ: thêm hoặc bớt một vài mẫu) cũng có thể dẫn đến việc cấu trúc cây được tạo ra hoàn toàn khác biệt. Điều này gây khó khăn cho việc giải thích mô hình một cách nhất quán.

1.4.3 4.3 Khó khăn với các mối quan hệ phi tuyến tính phức tạp

Decision Tree chia tách không gian dữ liệu bằng các đường thẳng song song với các trục tọa độ. Do đó, nó gặp khó khăn khi mô hình hóa các mối quan hệ phụ thuộc lẫn nhau phức tạp hoặc các đường biên quyết định dạng chéo/tròn.

1.4.4 4.4 Thiên kiến (Bias) đối với các biến có nhiều mức độ

Trong các bài toán có nhiều biến phân loại, Decision Tree có xu hướng ưu tiên chọn các thuộc tính có nhiều mức độ (nhiều giá trị khác nhau) để chia tách, ngay cả khi chúng không thực sự mang lại nhiều thông tin hữu ích.

1.4.5 4.5 Không tối ưu toàn cục

Thuật toán xây dựng cây thường dựa trên phương pháp tham lam (greedy algorithm) – đưa ra lựa chọn tốt nhất tại mỗi bước cục bộ. Điều này không đảm bảo tìm được cây tối ưu nhất trên toàn bộ cấu trúc.

Giải pháp: Để khắc phục các hạn chế này, người ta thường sử dụng các mô hình học kết hợp (Ensemble Learning) như **Random Forest** hoặc **Gradient Boosting Decision Trees (GBDT)** để tăng độ chính xác và tính ổn định.

1.5 5. Giới thiệu về Random Forest

Random Forest (Rừng ngẫu nhiên) là một thuật toán học máy mạnh mẽ thuộc nhóm **Ensemble Learning** (Học kết hợp). Ý tưởng chính của nó là xây dựng nhiều cây quyết định trong quá trình huấn luyện và kết hợp kết quả của chúng để đưa ra dự đoán cuối cùng.

1.5.1 5.1 Cách hoạt động

Random Forest sử dụng hai kỹ thuật chính để tạo ra sự khác biệt giữa các cây: 1. **Bagging (Bootstrap Aggregating):** Mỗi cây được huấn luyện trên một tập con ngẫu nhiên của dữ liệu (lấy mẫu có thay thế). 2. **Feature Randomness:** Tại mỗi lần chia tách nút, thuật toán chỉ chọn một nhóm ngẫu nhiên các đặc trưng (features) thay vì lấy tất cả, giúp các cây không quá giống nhau.

1.5.2 5.2 Tại sao dùng Random Forest?

- **Giảm Overfitting:** Nhờ việc trung bình cộng kết quả của nhiều cây, sai số do quá khớp trên một cây đơn lẻ sẽ bị triệt tiêu.
- **Độ chính xác cao:** Thường cho kết quả tốt hơn nhiều so với một Decision Tree duy nhất.
- **Độ ổn định:** Ít nhạy cảm hơn với các thay đổi nhỏ trong dữ liệu đầu vào.

Kết quả cuối cùng thường được quyết định bằng cách **biểu quyết (Voting)** cho bài toán phân loại hoặc **tính trung bình (Averaging)** cho bài toán hồi quy.

```
[9]: from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# 1. Khởi tạo mô hình Random Forest
# n_estimators: số lượng cây trong rừng (mặc định là 100)
```

```

rf_model = RandomForestClassifier(n_estimators=100, max_depth=2,
    ↪random_state=42)

# 2. Huấn luyện mô hình trên tập dữ liệu Iris
rf_model.fit(X_train, y_train)

# 3. Dự đoán và đánh giá
print(f"Độ chính xác của Random Forest trên tập kiểm tra: {rf_model.
    ↪score(X_test, y_test):.2f}")

# 4. Xem tầm quan trọng của các đặc trưng (Feature Importance)
# Đây là một ưu điểm lớn của Random Forest
for name, importance in zip(iris.feature_names, rf_model.feature_importances_):
    print(f"Đặc trưng {name}: {importance:.4f}")

```

Độ chính xác của Random Forest trên tập kiểm tra: 1.00
 Đặc trưng sepal length (cm): 0.1043
 Đặc trưng sepal width (cm): 0.0209
 Đặc trưng petal length (cm): 0.4124
 Đặc trưng petal width (cm): 0.4624

```

[10]: import matplotlib.pyplot as plt

# Tính toán accuracy cho cả hai mô hình trên toàn bộ tập dữ liệu Iris
dt_acc = clf.score(X_test, y_test)
rf_acc = rf_model.score(X_test, y_test)

print(f"Accuracy của Decision Tree: {dt_acc:.4f}")
print(f"Accuracy của Random Forest: {rf_acc:.4f}")

# Vẽ biểu đồ so sánh
models = ['Decision Tree', 'Random Forest']
accuracies = [dt_acc, rf_acc]

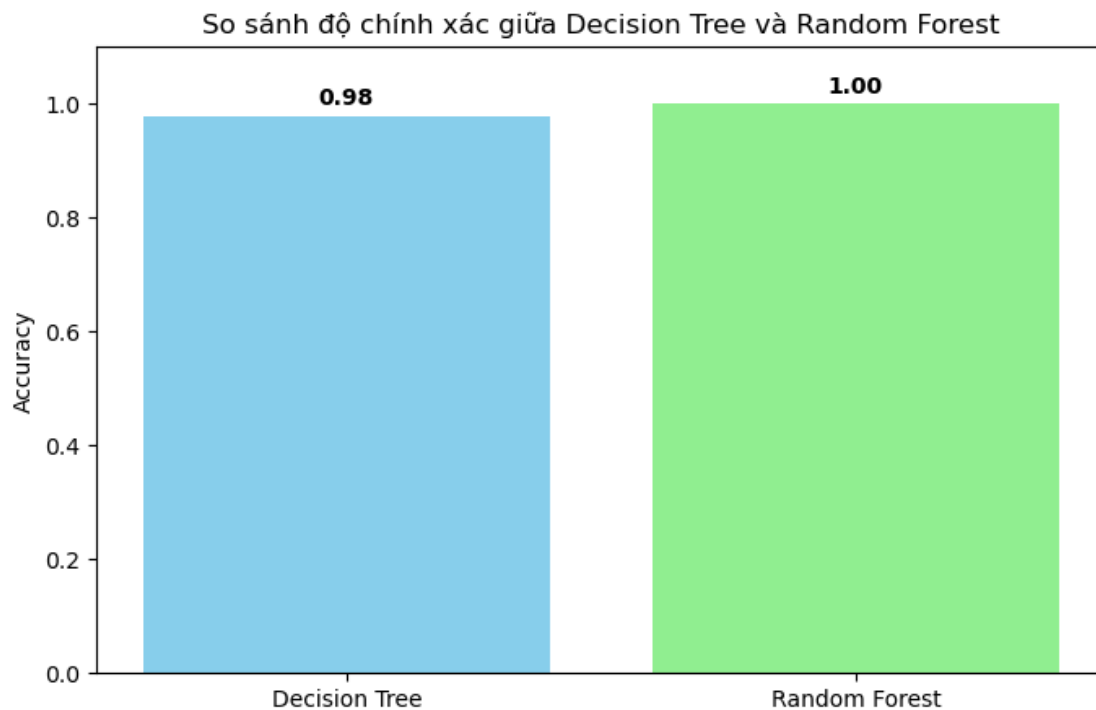
plt.figure(figsize=(8, 5))
plt.bar(models, accuracies, color=['skyblue', 'lightgreen'])
plt.ylim(0, 1.1)
plt.ylabel('Accuracy')
plt.title('So sánh độ chính xác giữa Decision Tree và Random Forest')

# Hiển thị giá trị trên cột
for i, acc in enumerate(accuracies):
    plt.text(i, acc + 0.02, f"{acc:.2f}", ha='center', fontweight='bold')

plt.show()

```

Accuracy của Decision Tree: 0.9778
 Accuracy của Random Forest: 1.0000



```
[11]: # sử dụng bộ dữ liệu wine
from sklearn.datasets import load_wine

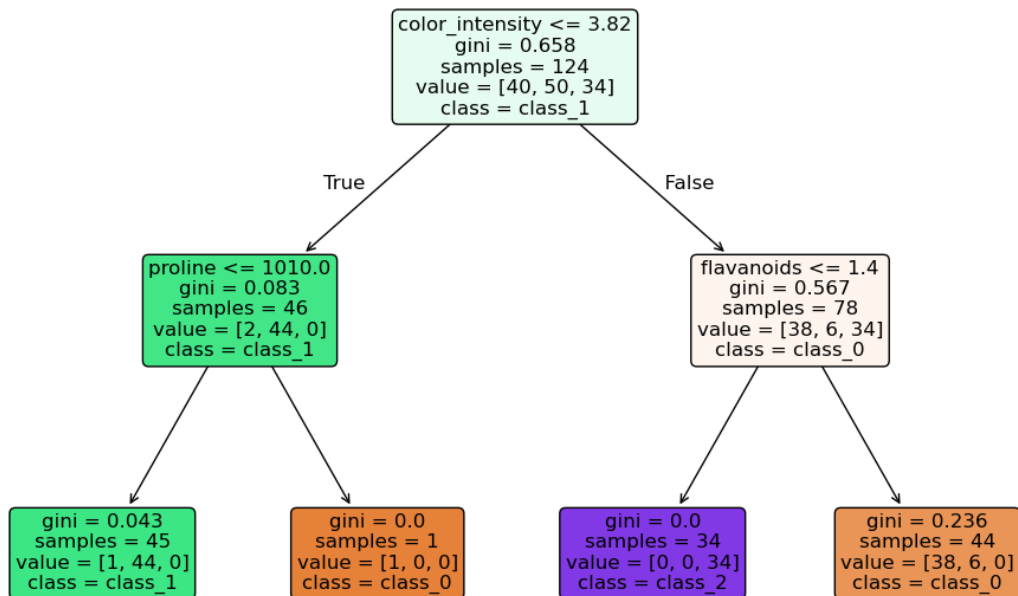
wine = load_wine()
X, y = wine.data, wine.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
↳random_state=42)
```

```
[12]: # 3. Khởi tạo và huấn luyện mô hình
dt_wine = DecisionTreeClassifier(max_depth=2, random_state=42)
dt_wine.fit(X_train, y_train)

# 3. Biểu diễn mô hình
plt.figure(figsize=(12, 8))
plot_tree(dt_wine,
          feature_names=wine.feature_names,
          class_names=wine.target_names.tolist(),
          filled=True,
          rounded=True,
          fontsize=12)
plt.title(f"Độ chính xác: {dt_wine.score(X_test, y_test):.4f}")
plt.show();
```

Độ chính xác: 0.8704



```
[13]: rf_wine = RandomForestClassifier(n_estimators=100, max_depth=2, random_state=42)

# 2. Huấn luyện mô hình trên tập dữ liệu Iris
rf_wine.fit(X_train, y_train)

# 3. Dự đoán và đánh giá
print(f"Độ chính xác của Random Forest trên tập kiểm tra: {rf_wine.\n↪score(X_test, y_test):.4f}")
```

Độ chính xác của Random Forest trên tập kiểm tra: 0.9630